A Matching Cut Reduction to 2-Cut on Line Graphs

Research manuscript · awaiting human expert review
15 September 2026

Abstract
The 2-Cut problem asks whether a graph admits a nontrivial vertex partition in which every vertex has at most two neighbors across the partition. We give a deterministic polynomial-time reduction from Matching Cut on connected graphs of maximum degree four to 2-Cut on connected line graphs. The construction uses a nine-edge root interface that forbids exactly the event that two neighbors oppose their common source vertex. Its constant-terminal property excludes nontrivial target cuts with constant source projections. For a source with n vertices, the root has at most $36n$ vertices and $69n$ edges. Every valid target cut yields a source matching cut by reading one edge per variable reservoir. Together with known source hardness, this establishes NP-completeness for the line-graph case left open in the cited literature.

1. Introduction
Restrictions on the input graph can change the complexity of finding a cut with bounded crossing degree. Matching Cut is the case with bound one. Chvátal proved its NP-completeness for graphs of maximum degree four [1]. Lucke et al. proved hardness of d-Cut on line graphs for $d \geq 3$, while their published conclusion explicitly retains the case $d = 2$ [2, Theorem 13 and Section 5]. Ahn et al. established 2-Cut hardness on claw-free graphs [3, Theorem 4]. That result does not supply the stronger line-graph restriction.

This paper gives an explicit root construction for the remaining case. We use the known principle that high-degree root vertices force a common incident-edge color. The required new interface has five private root vertices and three terminal edges. It represents one Matching Cut constraint and prevents a constant terminal assignment from hiding a nonconstant internal coloring.

Theorem 1. There is a deterministic polynomial-time Karp reduction from Matching Cut on connected simple graphs of maximum degree four to 2-Cut on connected simple line graphs. The reduction includes deterministic polynomial-time recovery of a source witness from every valid witness of its output.

The local interface and its composition are the mathematical content of the reduction. No novelty is claimed for clique rigidity or for encoding a constraint by a graph gadget. The literature assessment supporting the missing-case claim is recorded in the appendix, with its coverage limits.

2. Edge colors and monochromatic reservoirs
All graphs are finite, simple and undirected. A source witness for a graph G is a nonconstant map $x : V(G) \to \{0, 1\}$ satisfying

$$\sum_{u \in N_G(v)} [x(u) \neq x(v)] \leq 1 \quad \text{for every } v \in V(G). \tag{1}$$

Here brackets denote the indicator of a statement. A root graph H represents its line graph $L(H)$: root edges become vertices, and two vertices are adjacent exactly when their edges share an endpoint. A target witness is a nonconstant edge coloring $\chi : E(H) \to \{0, 1\}$ in which every edge has at most two adjacent edges of the other color.

For a root vertex u, let $b_t(u)$ count incident edges of color t. If uv has color t, its opposite adjacency count is

$$b_{1-t}(u) + b_{1-t}(v). \tag{2}$$

Simplicity ensures that the two sets being counted are disjoint. We allow constant colorings when discussing a local interface; nontriviality is imposed on the complete target.

Lemma 2 (mixed vertices). A vertex incident with both colors has degree at most four. If it has degree four, it has two edges of each color and every neighbor is monochromatic.

Proof. An incident edge of either color sees all incident edges of the other color, so neither color can occur more than twice. At degree four, each edge already sees two opposites at that endpoint. By Equation 2 its other endpoint has no incident edge of the opposite color. □

A *reservoir* is a copy of K_6, possibly with further edges attached to its vertices. Every reservoir vertex has degree at least five. By Lemma 2 it is monochromatic, and connectivity forces the whole reservoir and every incident attachment edge to share one bit. Attaching more edges preserves this property.

3. The three-terminal interface

Figure Figure 1 depicts the exact root interface. Its private vertices are $0, 1, 2, 3, 4$; external vertices A, B, C will belong to reservoirs. Write $\chi_{i,j}$ for the color of private edge ij, and let a, b, c be the colors of the terminal edges at $0, 1, 3$, respectively. The core edges, in fixed order, are

$$(01, 13, 14, 23, 24, 34), \tag{3}$$

where two juxtaposed vertex labels denote an edge. Each external endpoint is assumed monochromatic, and contributes zero to the opposite count of its terminal edge.

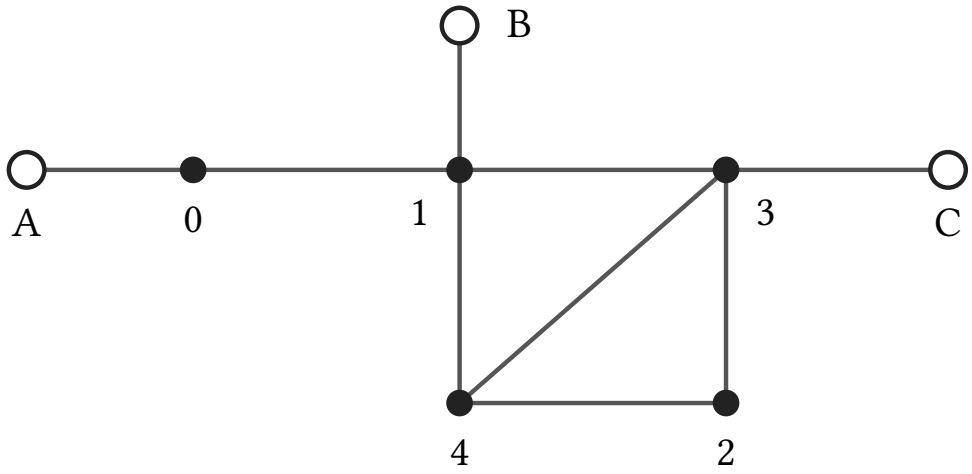


Figure 1: The exact nine-edge interface. Filled vertices form the private core; open vertices A, B, C are external endpoints. Terminal colors a, b, c belong to edges $0A, 1B, 3C$, respectively. A legal extension exists exactly when $a = c \neq b$ is false. Constant terminals force a constant core.

Lemma 3 (terminal relation). A terminal assignment extends to a legal local coloring if and only if

$$R(a, b, c) := \neg(a = c \wedge c \neq b). \tag{4}$$

If all three terminals have the same color, every core edge has that color.

Proof. First suppose $a = c = 0$ and $b = 1$. If vertex 1 were mixed, its neighbors $0, 3, 4$ would be monochromatic by Lemma 2. The terminals at 0 and 3 force $\chi_{0,1} = \chi_{1,3} = \chi_{3,4} = 0$; monochromatic vertex 4 then forces $\chi_{1,4} = 0$. Vertex 1 would have a forbidden three-to-one split. It is therefore monochromatic one. Vertex 3 is now mixed, so its neighbors 2 and 4 are monochromatic. The edge 14 forces 24 and 34 to one, and vertex 2 forces 23 to one. Vertex 3 has three incident ones against its zero terminal, again impossible. Complementing colors excludes $(1, 0, 1)$.

Now suppose all terminals are zero. If vertex 1 were mixed, the same neighbor argument would force $01, 13, 14$ all to zero. Thus vertex 1 is monochromatic zero. If vertex 3 were mixed, its edges 23 and

34 would both be one. Its neighbor 4 would then have to be monochromatic, contradicting $\chi_{1,4} = 0$ and $\chi_{3,4} = 1$. Thus vertex 3 is also monochromatic zero. The only remaining edge, 24, cannot be one: it would have two opposites at vertex 4 and one at vertex 2. Color complementation handles constant-one terminals.

Finally, Table Table 1 supplies a legal extension for each remaining terminal assignment. Each row satisfies Equation 2 for all nine edges. □

Terminal bits abc	Core bits in order $01, 13, 14, 23, 24, 34$
000	000000
001	011111
011	111111
100	000000
110	100000
111	111111

Table 1: Explicit extensions for all allowed terminal assignments. The two omitted assignments are excluded analytically in Lemma 3.

4. Construction and correctness

Let G be a connected source graph of maximum degree four with $n \geq 3$ vertices. Create one disjoint reservoir Q_v for each source vertex v. For every vertex b and every unordered pair of distinct neighbors a, c, create a fresh core from Lemma 3. Join its terminal vertices $0, 1, 3$ to fixed vertices of Q_a, Q_b, Q_c, respectively. Order $a < c$ to make this choice deterministic. Figure Figure 2 shows one clause with its three reservoirs.

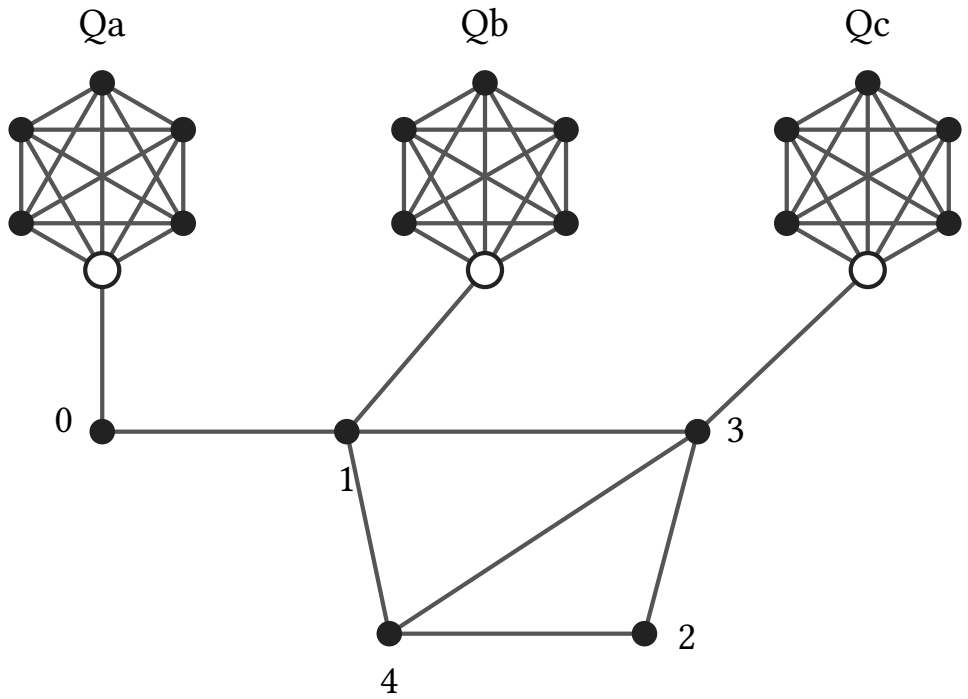


Figure 2: One complete clause attachment: three K_6 reservoirs and one private core, with all 54 edges shown. Open vertices are the reservoir attachment vertices. Further clauses may reuse those vertices; all their incident edges remain monochromatic. Labels Qa, Qb and Qc denote Q_a, Q_b, Q_c.

Lemma 4 (structural validity). The constructed root is simple and connected on every legal source input, regardless of its answer.

Proof. Cores are private, terminal edges have distinct private endpoints, and a, b, c are distinct source vertices. No loop or repeated edge is created. Every source edge in a connected graph with at least three vertices has an endpoint of degree at least two. It therefore occurs within some neighbor-pair clause, which connects its endpoint reservoirs. Since the source is connected, all reservoirs are connected through clauses. Every core is connected and joins its reservoirs. □

Proof of Theorem 1 for $n \geq 3$. Given a source matching cut x, color each reservoir and its terminal edges by $x(v)$. No source vertex has two opposite neighbors, so each terminal triple satisfies Equation 4. Extend every core using Table Table 1. All added costs at reservoir endpoints are zero, even when many clauses share a vertex. Private cores therefore compose without additional opposite adjacencies. Since x is nonconstant, this is a target witness.

Conversely, take any target witness. Reservoir rigidity assigns one bit $x(v)$ to each Q_v and fixes every terminal color. Lemma 3 excludes two neighbors opposing the same source vertex. Hence x meets all local Matching Cut constraints. If x were constant, all terminals of every clause would agree; the second part of Lemma 3 would make every private edge agree as well. The target would be constant, a contradiction. Thus x is a source witness. This argument applies to every valid target coloring. □

For the one-vertex source, output the root $K_{1,5}$, whose line graph K_5 has no 2-cut. For the only connected two-vertex source, output a two-edge path, whose line graph K_2 has a 2-cut; recover the source witness $(0, 1)$. These cases complete the theorem on legal inputs. Invalid source strings map to a fixed invalid target string, outside the target language.

5. Deterministic algorithms and bounds

Label reservoir vertices first, six per source vertex in source order. Output their edges first, in lexicographic order within each reservoir. Then process clauses in lexicographic order of (b, a, c), assigning five fresh labels and outputting the six core edges followed by the three terminal edges. The target is represented by its root and ordered edge list.

For zero-based source vertex v, position $15v$ of the target witness is the color of a fixed edge in Q_v. The recovery algorithm reads precisely those positions. It requires neither a solver nor state saved by the forward call. The proof above establishes the implication

$$R_T(f(G), y) \implies R_S(G, g(G, y)) \tag{5}$$

for every valid target witness y, where R_S and R_T are the source and target witness predicates. The exceptional two-vertex input uses its fixed witness; NO outputs carry no recovery obligation.

Let $T = \sum_{v \in V(G)} \binom{\deg(v)}{2}$ be the number of cores. For $n \geq 3$,

$$T \leq 6n, \quad |V(H)| = 6n + 5T \leq 36n, \quad |E(H)| = 15n + 9T \leq 69n. \tag{6}$$

There are linearly many bounded-degree source operations, and the root encoding uses $O(n \log(n+1))$ bits. An explicit line graph can instead be constructed with $O(n)$ vertices and $O(n^2)$ edges, preserving polynomial size.

All integers in the input are binary encoded. The implementation checks the schema, edge endpoints, degree and connectivity. Before allocating a vertex array, it checks $n \leq 2m + 1$, where m is the number of listed source edges. This necessary condition for connectedness prevents a binary vertex count from causing an exponentially large allocation. For an encoded input of length ℓ, a conservative worst-case bound for parsing, validation and construction is $O((\ell + 1)^3)$ bit operations. This bound does not assume expected constant-time hash-table operations. Recovery is polynomial in the combined source and witness encoding length and outputs n bits. Neither algorithm uses runtime randomness or an oracle.

6. Consequence

The source restriction to connected graphs preserves Chvátal's hardness [1]: disconnected nonempty graphs with at least two vertices are immediate YES instances and can be replaced by a fixed connected YES instance. The empty graph can be replaced by a fixed connected NO instance. Membership of the target in NP follows by checking an edge-color witness. Theorem 1 therefore gives NP-completeness for 2-Cut on line graphs, addressing the case retained in [2]. The argument depends on both the terminal relation and its constant-terminal property; a local relation alone would not establish nontrivial recovery.

References

[1] V. Chvátal. Recognizing decomposable graphs. *Journal of Graph Theory* 8(1), 51–53, 1984. doi:10.1002/jgt.3190080106.

[2] F. Lucke, A. Momeni, D. Paulusma and S. Smith. Finding d-Cuts in Graphs of Bounded Diameter, Graphs of Bounded Radius and H-Free Graphs. *Algorithmica* 88, Article 7, 2026. arXiv:2404.11389.

[3] J. Ahn, T. Eagling-Vose, F. Lucke, D. Paulusma and S. Smith. Finding d-Cuts in Claw-Free Graphs. *ISAAC 2025*, LIPIcs 359, Article 4. doi:10.4230/LIPIcs.ISAAC.2025.4.

A. Verification and reproducibility

The implementation is `algorithm.py`. Its forward mode reads an object with keys `n` and `edges`; each edge is an ordered pair of distinct zero-based vertices, with the smaller endpoint first. The output uses the same root encoding. Its edge-list order defines witness positions. Extraction reads `source` and `target_solution` and outputs the recovered source bit list. The precise contract is in `contract.md`.

The general proof is supported by finite checks, not inferred from them. The preparation suite compared Z3 with exhaustive enumeration on 31 source graphs and 29 target roots through five vertices, and rejected three deliberately bad programs. The complete candidate passed eight prepared cases with 22 recoveries and a separate 95-case suite with 106 recoveries. The latter covers 34 YES and 61 NO inputs, including all 64 subsets of six length-two chords added to a six-cycle. Its largest root has 216 vertices and 414 edges.

An independent reviewer checked all 512 local assignments, five additional complete instances and six recoveries. The largest of those roots has 288 vertices and 552 edges. On three nonspecial outputs, independent solver queries excluded every legal nonconstant target coloring with a constant or invalid source projection. These are universal checks on particular finite outputs; the proof in the main text supplies the arbitrary-input argument.

From this directory, with Python 3, NetworkX and Z3 5.1.0 installed:

```
python3 check.py --self-test
python3 check.py --candidate algorithm.py
python3 verify.py --candidate algorithm.py
python3 algorithm.py < source.json > target.json
python3 algorithm.py --extract < recovery-input.json
```

`cases.json` contains the stored source cases. Detailed outputs are retained in `../rounds/003/` and `../reviews/complete-rule/`. The local search used 740 core/terminal templates and 315,272 edge assignments; it found six interfaces with the required relation. Only the first was used in this reduction. The candidate itself uses the standard library and performs no search.

The literature check dated 15 September 2026 inspected [2], including the published Theorem 13 and Section 5, and [3], Theorem 4. It checked exact and edge-coloring formulations and searched for later line-graph resolutions. The publisher record of [1] was inspected; its full proof was not retrieved. No exact prior resolution was located within this coverage. Unpublished, unindexed or overlooked work remains outside that assessment. The independent review returned advance; human expert assessment remains outstanding.